Informe de práctica de la semana 4

Interfaces composición e inyección de dependencias

UNAP · Ingeniería de Sistemas · SIS206 POO II · 2026 II
S4-001 · 19 de octubre de 2026 · Informe revisado

1 Propósito y actividades

Definir contratos intercambiables para notificaciones y reportes, y separar la cotización del hotel mediante una dependencia recibida desde el exterior.

Los esquemas mezclan contratos con métodos pendientes y exportaciones simuladas. Se completaron las implementaciones, se aislaron errores por canal y se añadió una prueba con exportador de memoria. Las notificaciones continúan siendo simulaciones y no contactan servicios externos.

Actividades desarrolladas

Actividad	Archivo o componente	Resultado
1 Notificaciones	python/solucion.py	ABC, Protocol, Email, SMS y SistemaAlertas.
2 Reportes	python/solucion.py	Cambio de exportador, firma simulada y mock verificable.
3 C++	cpp/solucion.hpp	Interfaz abstracta y propiedad mediante unique_ptr.
4 Comparaciones	ACTIVIDADES_RESUELTAS.md	Tablas de abstracción y estrategias de diseño completas.
5 Hotel	CalculadoraTarifa y ServicioCotizacion	Interfaz de dos métodos y cálculo recibido en el constructor.

Caso explicado

La habitación doble tiene un precio de S/ 150 por noche. ServicioCotizacion recibe una TarifaEstandar y solicita una cotización de dos noches; el resultado es S/ 300. La fórmula se encuentra en la calculadora recibida, de modo que el servicio puede trabajar con otra implementación del mismo contrato.

Conceptos para interpretar el caso

Una interfaz define operaciones que un colaborador debe ofrecer. La inyección de dependencias consiste en recibir ese colaborador desde fuera. Un mock es un sustituto de prueba que permite comprobar llamadas sin depender del recurso real.

La solución y sus pruebas se encuentran en desarrollo. Las respuestas completas y el notebook amplían los ejemplos de este informe.

2 Desarrollo en Python y C++

Email y SMS implementan INotificador y comparten un CanalSimulado por composición. Un SMS no hereda de Email. SistemaAlertas cuenta solo envíos exitosos y registra el error de un canal sin impedir que los demás se procesen.

GeneradorReporte recibe exportador y firmador opcional. En Python transforma el diccionario a una representación intermedia y sus exportadores escriben CSV o JSON válido. En C++ recibe contenido ya serializado y delega su destino; esta diferencia del contrato interno se declara, y no se afirma una comparación byte a byte de los archivos exportados.

FirmadorDigital es una simulación explícita que crea una copia marcada. No ofrece autenticidad, certificados ni una firma criptográfica. Los mensajes de notificación se guardan en memoria; no se utiliza correo ni telefonía reales.

Contratos y estrategias de diseño

Criterio	Python	C++ o efecto en el diseño
Instanciación	ABC abstracta y Protocol no se instancian	Clase con virtual puro no se instancia
Estado e implementación	ABC puede tener ambos; Protocol describe estructura	Una clase abstracta puede tener ambos; una interfaz pura los evita por diseño
Método abstracto	@abstractmethod	virtual metodo(...) = 0
Herencia explícita	ABC nominal; Protocol puede ser estructural	Implementación nominal mediante herencia
Cambio de comportamiento	Composición permite sustituir dependencias	Herencia fija la estructura del subtipo
Pruebas	Mock o doble inyectado sin archivos	Implementación de interfaz recibida por el cliente

Cómo seguir la implementación

Python expresa contratos mediante ABC y también mediante Protocol estructural. C++ los representa con funciones virtuales puras. El constructor recibe el colaborador y la operación pública delega en él. La composición permite sustituir el comportamiento sin copiar el cliente ni obligar a SMS a heredar de Email.

3 Verificación y resultados

Las pruebas registradas aprobaron 11 casos Python y 10 casos C++17. Se ejecutaron en Python 3.12.14 y g++ 10.3.0. Las demostraciones de ambos lenguajes producen la misma salida.

Comprobación	Resultado	Registro
Pruebas Python	11 aprobadas	evidencias/pruebas_python.txt
Pruebas C++17	10 aprobadas	evidencias/pruebas_cpp.txt
Compilación y demostraciones	Sin advertencias y salidas iguales	evidencias/verificacion.json

Pytest ejecutó los mismos casos que unittest; sus resultados no se suman como pruebas diferentes. Las compilaciones usaron -Wall, -Wextra, -Wpedantic y -Werror.

Casos representativos comprobados

Caso	Resultado observado	Qué demuestra
Email y SMS disponibles	Dos envíos simulados exitosos	El cliente usa ambos canales por su contrato.
SMS de 161 caracteres y Email válido	Un éxito y un error registrado	El fallo de un canal no interrumpe al siguiente.
Generación con ExportadorMock	Una llamada con el contenido esperado	Se comprueba la colaboración sin escribir archivos.
Habitación doble por dos noches	Cotización de S/ 300	La tarifa inyectada calcula el total.

La verificación incluye sustitución del colaborador y recuperación de errores. La coincidencia de las demostraciones no implica que ambos exportadores compartan el mismo contrato interno: Python serializa datos y la versión C++ recibe contenido ya serializado.

Lectura de la salida

CANALES=2 indica dos envíos simulados exitosos. SMS_LARGO=1 corresponde al caso con un canal fallido y otro exitoso. COTIZACION=300.00 es el precio de dos noches en la habitación doble.

4 Aplicación al hotel y conclusiones

CalculadoraTarifa declara calcular y descripcion. TarifaEstandar implementa el contrato; ServicioCotizacion recibe esa dependencia. Se utiliza herencia para representar el contrato y composición para elegir el comportamiento de cotización. El diagrama distingue propiedad exclusiva, referencias y realización de interfaz.

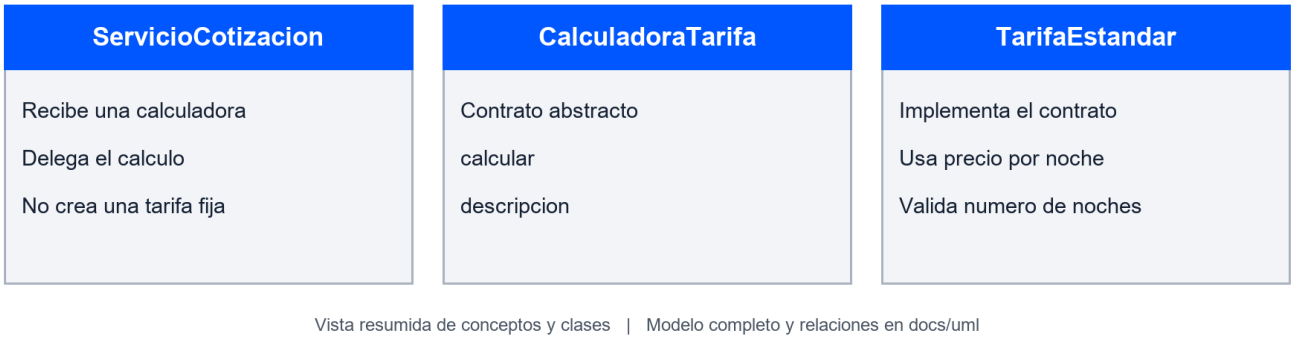


Figura 1 Esquema de los conceptos aplicados al hotel. El diagrama de clases completo se encuentra en docs/uml.

Conclusión de la práctica

El servicio quedó separado del cálculo y del canal de salida. Las pruebas demuestran que se pueden sustituir colaboradores y aislar fallos. Las notificaciones y la firma siguen siendo simulaciones explícitas, útiles para estudiar el diseño sin contactar servicios externos.

Para comprobar la comprensión, seguir el caso explicado en los dos lenguajes e identificar la validación, el cambio de estado y la prueba que lo verifica. El anexo ACTIVIDADES_RESUELTAS.md desarrolla las preguntas de reflexión.

Transparencia y datos de entrega

Se utilizó Codex para analizar el material, desarrollar y verificar el código y preparar los documentos. Antes de entregar, completar los datos del estudiante y GitHub. La revisión de pares y la sustentación requieren participación humana.

Referencias

Fowler, M. (2018). *Refactoring Improving the Design of Existing Code* (2.ª ed.). Addison Wesley. https://martinfowler.com/books/refactoring.html

Object Management Group. (2017). *OMG Unified Modeling Language Version 2.5.1*. https://www.omg.org/spec/UML/2.5.1/

Python Software Foundation. (s. f.). *abc — Abstract Base Classes*. Python 3.12 documentation. https://docs.python.org/3.12/library/abc.html

Zanabria Gálvez, A. H. (2026). *Guía de práctica de laboratorio de la semana 4*. SIS206 POO II, Universidad Nacional del Altiplano. Material del curso.